

4. INFORMATION-BASED LEARNING (2)

SECTIONS 4.4~4.5



CONTENTS-2

- **Alternative Feature Selection Metrics**
- **Handling Continuous Descriptive Features**
- **Predicting Continuous Targets**
- **Noisy Data, Overfitting and Tree Pruning**
- **Model Ensembles**
 - Bagging
 - Boosting
- **Summary**

INFORMATION GAIN RATIO

- Entropy based information gain **preferences** features with many values.
- One way of addressing this issue is to use **information gain ratio** which is computed by dividing the information gain of a feature by the amount of information used to determine the value of the feature:

$$GR(d, D) = \frac{IG(d, D)}{- \sum_{l \in \text{levels}(d)} P(d=l) \times \log_2 P(d=l)} \quad (4.8)$$

A WORKED EXAMPLE: PREDICTING VEGETATION DISTRIBUTIONS

- **Table 4.3:** The vegetation classification dataset.

ID	STREAM	SLOPE	ELEVATION	VEGETATION
1	false	steep	high	chaparral
2	true	moderate	low	riparian
3	true	steep	medium	riparian
4	false	steep	medium	chaparral
5	false	flat	high	conifer
6	true	steep	highest	conifer
7	true	steep	high	chaparral

A WORKED EXAMPLE: PREDICTING VEGETATION DISTRIBUTIONS

Table 4.4: Partition sets (Part.), entropy, remainder (Rem.) and information gain (Info. Gain) by feature for the dataset in Table 4.3

Split By Feature	Level	Part.	Instances	Partition Entropy	Rem.	Info. Gain
STREAM	'true'	$\mathcal{D}_1$	$\mathbf{d}_2, \mathbf{d}_3, \mathbf{d}_6, \mathbf{d}_7$	1.5	1.2507	0.3060
	'false'	$\mathcal{D}_2$	$\mathbf{d}_1, \mathbf{d}_4, \mathbf{d}_5$	0.9183		
SLOPE	'flat'	$\mathcal{D}_3$	$\mathbf{d}_5$	0	0.9793	0.5774
	'moderate'	$\mathcal{D}_4$	$\mathbf{d}_2$	0		
	'steep'	$\mathcal{D}_5$	$\mathbf{d}_1, \mathbf{d}_3, \mathbf{d}_4, \mathbf{d}_6, \mathbf{d}_7$	1.3710		
ELEVATION	'low'	$\mathcal{D}_6$	$\mathbf{d}_2$	0	0.6793	0.8774
	'medium'	$\mathcal{D}_7$	$\mathbf{d}_3, \mathbf{d}_4$	1.0		
	'high'	$\mathcal{D}_8$	$\mathbf{d}_1, \mathbf{d}_5, \mathbf{d}_7$	0.9183		
	'highest'	$\mathcal{D}_9$	$\mathbf{d}_6$	0		

ID	STREAM	SLOPE	ELEVATION	VEGETATION
1	false	steep	high	chaparral
2	true	moderate	low	riparian
3	true	steep	medium	riparian
4	false	steep	medium	chaparral
5	false	flat	high	conifer
6	true	steep	highest	conifer
7	true	steep	high	chaparral

$$H(\text{STREAM}, D)$$

$$= - \sum_{l \in \{'true', 'false'\}} P(\text{STREAM} = l) \times \log_2 P(\text{STREAM} = l)$$

$$= - \left[\frac{4}{7} \log_2 \frac{4}{7} + \frac{3}{7} \log_2 \frac{3}{7} \right] = 0.9852 \text{ bits}$$

$$H(\text{SLOPE}, D)$$

$$= - \sum_{l \in \{'flat', 'moderate', 'steep'\}} P(\text{SLOPE} = l) \times \log_2 P(\text{SLOPE} = l)$$

$$= - \left[\frac{1}{7} \log_2 \frac{1}{7} + \frac{1}{7} \log_2 \frac{1}{7} + \frac{5}{7} \log_2 \frac{5}{7} \right] = 1.1488 \text{ bits}$$

$$H(\text{ELEVATION}, D)$$

$$= - \sum_{l \in \{'low', 'medium', 'high', 'highest'\}} P(\text{ELEVATION} = l) \times \log_2 P(\text{ELEVATION} = l)$$

$$= - \left[\frac{1}{7} \log_2 \frac{1}{7} + \frac{2}{7} \log_2 \frac{2}{7} + \frac{3}{7} \log_2 \frac{3}{7} + \frac{1}{7} \log_2 \frac{1}{7} \right] = 1.8424 \text{ bits}$$

$$GR(\text{STREAM}, D) = \frac{0.3060}{0.9852} = 0.3106$$

$$GR(\text{SLOPE}, D) = \frac{0.5774}{1.1488} = 0.5026$$

$$GR(\text{ELEVATION}, D) = \frac{0.8774}{1.8424} = 0.4762$$

ID	STREAM	SLOPE	ELEVATION	VEGETATION
1	false	steep	high	chaparral
2	true	moderate	low	riparian
3	true	steep	medium	riparian
4	false	steep	medium	chaparral
5	false	flat	high	conifer
6	true	steep	highest	conifer
7	true	steep	high	chaparral

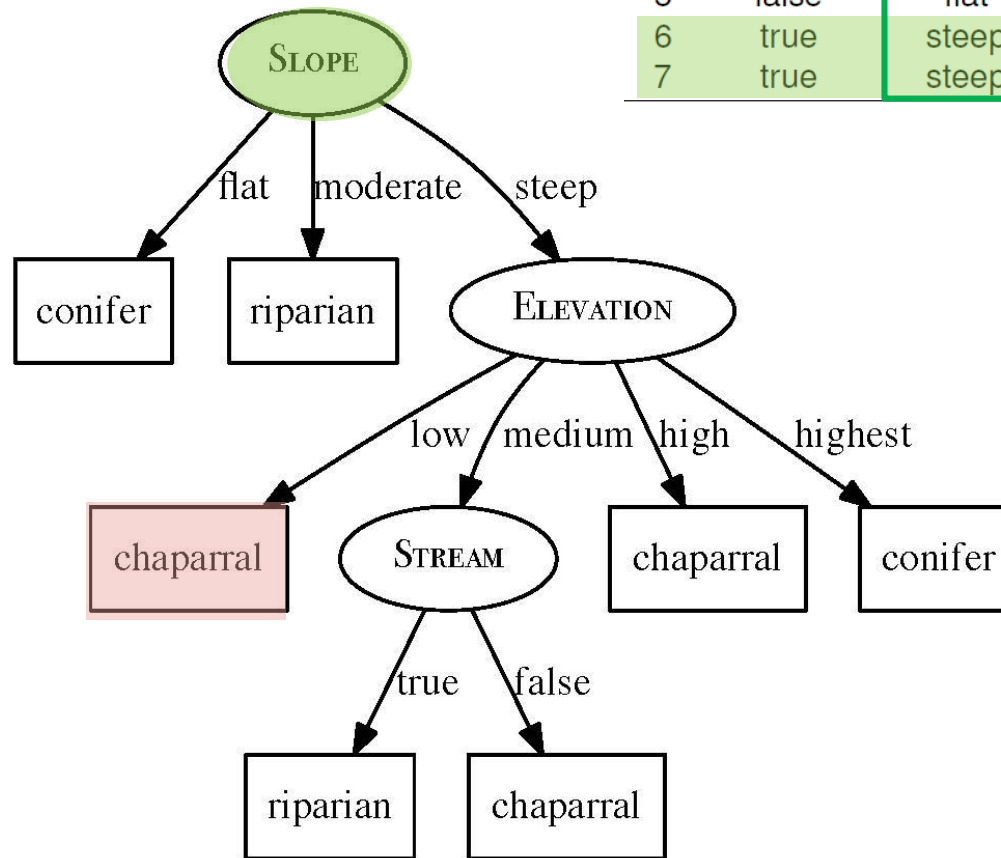


Figure4.12: The vegetation classification decision tree generated using **information gain ratio**

$$\frac{1+1+2+3+3+2+2}{7} = \frac{14}{7} = 2$$

$$\frac{1+2+2+2+2+2+1}{7} = \frac{12}{7}$$

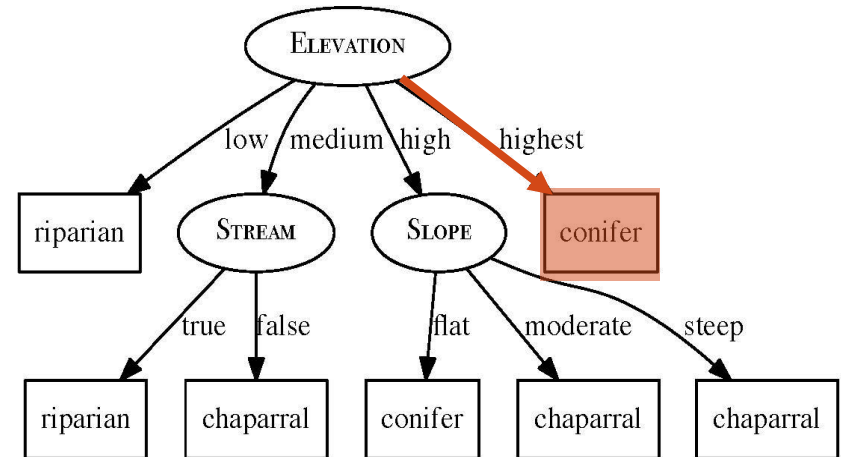
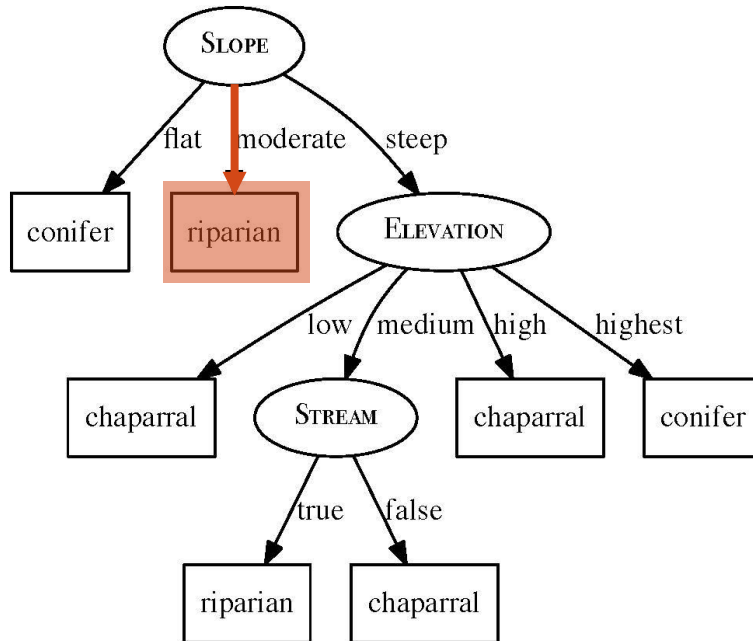


Figure4.12 vs Figure4.11

STREAM = false, SLOPE = moderate, ELEVATION = highest

- The combination of features listed in this query does not occur in the dataset.
STREAM = false, SLOPE = moderate, ELEVATION = highest
- Consequently, both of the trees are attempting to generalize beyond the dataset.
- This illustrates how two **different models** that are both consistent with a dataset can make **different generalizations**.

- Which feature selection metric should be used, **information gain** or **information gain ratio**?
- **Information gain** has the advantage that it is **computationally less expensive** than information gain ratio.
- If there is **variation across the number of values** in the domain of the descriptive features in a dataset, however, **information gain ratio** may be a better option.
- The **effectiveness** of descriptive feature selection metrics can **vary from domain to domain**.
- We should experiment with different metrics to find which one results in the best models for each dataset.

GINI INDEX

- Another commonly used measure of impurity is the **Gini index**:

$$Gini(t, D) = 1 - \sum_{l \in \text{levels}(t)} P(t = l)^2 \quad (4.9)$$

- Information gain can be calculated using the Gini index by **replacing** the **entropy** measure with the **Gini index**.
- Gini index scores are always **between 0 and 1 (?)**, this may make it easier to compare Gini indexes across features.

GINI INDEX

- The Gini index can be thought of as calculating how often you would **misclassify** an instance in the dataset if you classified it based on the distribution of classifications in the dataset.
 - if there were two target levels with equal likelihood in a dataset, then the expected rate of misclassification would be 0.5
 - if there were four target levels with equal likelihood, then the expected rate of misclassification would be 0.75
 - Gini index=0 when all the instances in the dataset have the same target level
 - Gini index= $1 - 1/k$ when there are k possible target levels with equal likelihood

Table 4.3: The vegetation classification dataset.

ID	STREAM	SLOPE	ELEVATION	VEGETATION
1	false	steep	high	chaparral
2	true	moderate	low	riparian
3	true	steep	medium	riparian
4	false	steep	medium	chaparral
5	false	flat	high	conifer
6	true	steep	highest	conifer
7	true	steep	high	chaparral

$$Gini(\text{VEGETATION}, D)$$

$$= 1 - \sum_{l \in \left\{ \begin{array}{l} \text{'chaparral'}, \\ \text{'riparian'}, \\ \text{'conifer'} \end{array} \right\}} P(\text{VEGETATION} = l)^2$$

$$= 1 - \left[\left(\frac{3}{7} \right)^2 + \left(\frac{2}{7} \right)^2 + \left(\frac{2}{7} \right)^2 \right]$$

$$= 0.6531$$

Feature Selection		Cont. Desc. Features		Cont. Targets	Noise and Overfitting	Ensembles	Summary
ID	STREAM	SLOPE	ELEVATION	VEGETATION	CONTRIBUTIONS		
1	false	steep	high	chaparral			
2	true	moderate	low	riparian			
3	true	steep	medium	riparian			
4	false	steep	medium	chaparral			
5	false	flat	high	conifer			
6	true	steep	highest	conifer			
7	true	steep	high	chaparral			

remainder (Rem.), and

Split by Feature	Level	Part.	Instances	Partition Gini Index	Rem.	Info. Gain
STREAM	'true'	$\mathcal{D}_1$	$\mathbf{d}_2, \mathbf{d}_3, \mathbf{d}_6, \mathbf{d}_7$	0.625	0.5476	0.1054
	'false'	$\mathcal{D}_2$	$\mathbf{d}_1, \mathbf{d}_4, \mathbf{d}_5$	0.4444		
SLOPE	'flat'	$\mathcal{D}_3$	$\mathbf{d}_5$	0	0.4	0.2531
	'moderate'	$\mathcal{D}_4$	$\mathbf{d}_2$	0		
	'steep'	$\mathcal{D}_5$	$\mathbf{d}_1, \mathbf{d}_3, \mathbf{d}_4, \mathbf{d}_6, \mathbf{d}_7$	0.56		
ELEVATION	'low'	$\mathcal{D}_6$	$\mathbf{d}_2$	0	0.3333	0.3198
	'medium'	$\mathcal{D}_7$	$\mathbf{d}_3, \mathbf{d}_4$	0.5		
	'high'	$\mathcal{D}_8$	$\mathbf{d}_1, \mathbf{d}_5, \mathbf{d}_7$	0.4444		
	'highest'	$\mathcal{D}_9$	$\mathbf{d}_6$	0		

ID	STREAM	SLOPE	ELEVATION	VEGETATION
1	false	steep	high	chaparral
2	true	moderate	low	riparian
3	true	steep	medium	riparian
4	false	steep	medium	chaparral
5	false	flat	high	conifer
6	true	steep	highest	conifer
7	true	steep	high	chaparral

CONTRIBUTIONS

remainder (Rem.), and

Split by Feature	Level	Part.	Instances	Partition Gini Index	Rem.	Info. Gain
STREAM	'true'	$\mathcal{D}_1$	$\mathbf{d}_2, \mathbf{d}_3, \mathbf{d}_6, \mathbf{d}_7$	0.625	0.5476	0.1054
	'false'	$\mathcal{D}_2$	$\mathbf{d}_1, \mathbf{d}_4, \mathbf{d}_5$	0.4444		
SLOPE	'flat'	$\mathcal{D}_3$	$\mathbf{d}_5$	0	0.4	0.2531
	'moderate'	$\mathcal{D}_4$	$\mathbf{d}_2$	0		
	'steep'	$\mathcal{D}_5$	$\mathbf{d}_1, \mathbf{d}_3, \mathbf{d}_4, \mathbf{d}_6, \mathbf{d}_7$	0.56		
ELEVATION	'low'	$\mathcal{D}_6$	$\mathbf{d}_2$	0	0.3333	0.3198
	'medium'	$\mathcal{D}_7$	$\mathbf{d}_3, \mathbf{d}_4$	0.5		
	'high'	$\mathcal{D}_8$	$\mathbf{d}_1, \mathbf{d}_5, \mathbf{d}_7$	0.4444		
	'highest'	$\mathcal{D}_9$	$\mathbf{d}_6$	0		

ID	STREAM	SLOPE	ELEVATION	VEGETATION
1	false	steep	high	chaparral
2	true	moderate	low	riparian
3	true	steep	medium	riparian
4	false	steep	medium	chaparral
5	false	flat	high	conifer
6	true	steep	highest	conifer
7	true	steep	high	chaparral

CONTRIBUTIONS

remainder (Rem.), and

Split by Feature	Level	Part.	Instances	Partition Gini Index	Rem.	Info. Gain
STREAM	'true'	$\mathcal{D}_1$	$\mathbf{d}_2, \mathbf{d}_3, \mathbf{d}_6, \mathbf{d}_7$	0.625	0.5476	0.1054
	'false'	$\mathcal{D}_2$	$\mathbf{d}_1, \mathbf{d}_4, \mathbf{d}_5$	0.4444		
SLOPE	'flat'	$\mathcal{D}_3$	$\mathbf{d}_5$	0	0.4	0.2531
	'moderate'	$\mathcal{D}_4$	$\mathbf{d}_2$	0		
	'steep'	$\mathcal{D}_5$	$\mathbf{d}_1, \mathbf{d}_3, \mathbf{d}_4, \mathbf{d}_6, \mathbf{d}_7$	0.56		
ELEVATION	'low'	$\mathcal{D}_6$	$\mathbf{d}_2$	0	0.3333	0.3198
	'medium'	$\mathcal{D}_7$	$\mathbf{d}_3, \mathbf{d}_4$	0.5		
	'high'	$\mathcal{D}_8$	$\mathbf{d}_1, \mathbf{d}_5, \mathbf{d}_7$	0.4444		
	'highest'	$\mathcal{D}_9$	$\mathbf{d}_6$	0		

- **Table4.7:** Partition sets (Part.), Gini index, remainder (Rem.), and information gain (Info. Gain) by feature

Split by Feature	Level	Part.	Instances	Partition Gini Index	Rem.	Info. Gain
STREAM	'true'	$\mathcal{D}_1$	$\mathbf{d}_2, \mathbf{d}_3, \mathbf{d}_6, \mathbf{d}_7$	0.625	0.5476	0.1054
	'false'	$\mathcal{D}_2$	$\mathbf{d}_1, \mathbf{d}_4, \mathbf{d}_5$	0.4444		
SLOPE	'flat'	$\mathcal{D}_3$	$\mathbf{d}_5$	0	0.4	0.2531
	'moderate'	$\mathcal{D}_4$	$\mathbf{d}_2$	0		
	'steep'	$\mathcal{D}_5$	$\mathbf{d}_1, \mathbf{d}_3, \mathbf{d}_4, \mathbf{d}_6, \mathbf{d}_7$	0.56		
ELEVATION	'low'	$\mathcal{D}_6$	$\mathbf{d}_2$	0	0.3333	0.3198
	'medium'	$\mathcal{D}_7$	$\mathbf{d}_3, \mathbf{d}_4$	0.5		
	'high'	$\mathcal{D}_8$	$\mathbf{d}_1, \mathbf{d}_5, \mathbf{d}_7$	0.4444		
	'highest'	$\mathcal{D}_9$	$\mathbf{d}_6$	0		

- Comparing these results to the **information gain** calculated using entropy (see Table 4.4), we can see that although the resulting numbers are different, **the relative ranking** of the features is the same.

Table 4.4

Split By Feature	Level	Part.	Instances	Partition Entropy	Rem.	Info. Gain
STREAM	'true'	$\mathcal{D}_1$	$\mathbf{d}_2, \mathbf{d}_3, \mathbf{d}_6, \mathbf{d}_7$	1.5	1.2507	0.3060
	'false'	$\mathcal{D}_2$	$\mathbf{d}_1, \mathbf{d}_4, \mathbf{d}_5$	0.9183		
SLOPE	'flat'	$\mathcal{D}_3$	$\mathbf{d}_5$	0	0.9793	0.5774
	'moderate'	$\mathcal{D}_4$	$\mathbf{d}_2$	0		
	'steep'	$\mathcal{D}_5$	$\mathbf{d}_1, \mathbf{d}_3, \mathbf{d}_4, \mathbf{d}_6, \mathbf{d}_7$	1.3710		
ELEVATION	'low'	$\mathcal{D}_6$	$\mathbf{d}_2$	0	0.6793	0.8774
	'medium'	$\mathcal{D}_7$	$\mathbf{d}_3, \mathbf{d}_4$	1.0		
	'high'	$\mathcal{D}_8$	$\mathbf{d}_1, \mathbf{d}_5, \mathbf{d}_7$	0.9183		
	'highest'	$\mathcal{D}_9$	$\mathbf{d}_6$	0		

Table 4.7

Split by Feature	Level	Part.	Instances	Partition Gini Index	Rem.	Info. Gain
STREAM	'true'	$\mathcal{D}_1$	$\mathbf{d}_2, \mathbf{d}_3, \mathbf{d}_6, \mathbf{d}_7$	0.625	0.5476	0.1054
	'false'	$\mathcal{D}_2$	$\mathbf{d}_1, \mathbf{d}_4, \mathbf{d}_5$	0.4444		
SLOPE	'flat'	$\mathcal{D}_3$	$\mathbf{d}_5$	0	0.4	0.2531
	'moderate'	$\mathcal{D}_4$	$\mathbf{d}_2$	0		
	'steep'	$\mathcal{D}_5$	$\mathbf{d}_1, \mathbf{d}_3, \mathbf{d}_4, \mathbf{d}_6, \mathbf{d}_7$	0.56		
ELEVATION	'low'	$\mathcal{D}_6$	$\mathbf{d}_2$	0	0.3333	0.3198
	'medium'	$\mathcal{D}_7$	$\mathbf{d}_3, \mathbf{d}_4$	0.5		
	'high'	$\mathcal{D}_8$	$\mathbf{d}_1, \mathbf{d}_5, \mathbf{d}_7$	0.4444		
	'highest'	$\mathcal{D}_9$	$\mathbf{d}_6$	0		

- For the vegetation dataset, the decision tree that will be generated using information gain based on **the Gini index** will **be identical to** the one generated using information gain based on **entropy**.
- Which impurity measure should be used, **Gini** or **entropy**?
- It is good practice when building decision tree models to **try out** different impurity metrics and **compare** the results to see which suits a dataset best.

CONTENTS

- **Alternative Feature Selection Metrics**
- **Handling Continuous Descriptive Features**
- **Predicting Continuous Targets**
- **Noisy Data, Overfitting and Tree Pruning**
- **Model Ensembles**
 - Bagging
 - Boosting
- **Summary**

- The easiest way to handle continuous valued descriptive features is to turn them into **boolean features** by defining a threshold and using this threshold to partition the instances based their value of the continuous descriptive feature.
- How do we set **the threshold**?

- The instances in the dataset **are sorted** according to the continuous feature values.
- The adjacent instances in the ordering that have **different classifications** are then selected as possible threshold points.
- The **optimal threshold** is found by computing the information gain for each of these classification transition boundaries and selecting the boundary with **the highest information gain** as the threshold.

- Once a threshold has been set, the **dynamically created** new boolean feature can compete with the other categorical features for selection as the splitting feature at that node.
- This process can be repeated at each node as the tree grows.

Table 4.8: Dataset for predicting the vegetation in an area with a **continuous ELEVATION** feature (measured in feet).

ID	STREAM	SLOPE	ELEVATION	VEGETATION
1	false	steep	3 900	chapparal
2	true	moderate	300	riparian
3	true	steep	1 500	riparian
4	false	steep	1 200	chapparal
5	false	flat	4 450	conifer
6	true	steep	5 000	conifer
7	true	steep	3 000	chapparal

Table 4.9: Dataset for predicting the vegetation in an area sorted by the continuous ELEVATION feature.

ID	STREAM	SLOPE	ELEVATION	VEGETATION
2	true	moderate	300	riparian
4	false	steep	1 200	chapparal
3	true	steep	1 500	riparian
7	true	steep	3 000	chapparal
1	false	steep	3 900	chapparal
5	false	flat	4 450	conifer
6	true	steep	5 000	conifer

the boundary between **d**₂ and **d**₄ is $\frac{300+1200}{2}=750$

the boundary between **d**₄ and **d**₃ is $\frac{1200+1500}{2}=1350$

the boundary between **d**₃ and **d**₇ is $\frac{1500+3000}{2}=2250$

the boundary between **d**₁ and **d**₅ is $\frac{3900+4450}{2}=4175$

Table: Partition sets (Part.), entropy, remainder (Rem.), and information gain (Info. Gain) for the candidate ELEVATION thresholds: ≥ 750 , ≥ 1350 , ≥ 2250 and ≥ 4175

Split by Threshold	Part.	Instances	Partition Entropy	Rem.	Info. Gain
≥ 750	$\mathcal{D}_1$	$\mathbf{d}_2$	0.0	1.2507	0.3060
	$\mathcal{D}_2$	$\mathbf{d}_4, \mathbf{d}_3, \mathbf{d}_7, \mathbf{d}_1, \mathbf{d}_5, \mathbf{d}_6$	1.4591		
≥ 1350	$\mathcal{D}_3$	$\mathbf{d}_2, \mathbf{d}_4$	1.0	1.3728	0.1839
	$\mathcal{D}_4$	$\mathbf{d}_3, \mathbf{d}_7, \mathbf{d}_1, \mathbf{d}_5, \mathbf{d}_6$	1.5219		
≥ 2250	$\mathcal{D}_5$	$\mathbf{d}_2, \mathbf{d}_4, \mathbf{d}_3$	0.9183	0.9650	0.5917
	$\mathcal{D}_6$	$\mathbf{d}_7, \mathbf{d}_1, \mathbf{d}_5, \mathbf{d}_6$	1.0		
≥ 4175	$\mathcal{D}_7$	$\mathbf{d}_2, \mathbf{d}_4, \mathbf{d}_3, \mathbf{d}_7, \mathbf{d}_1$	0.9710	0.6935	0.8631
	$\mathcal{D}_8$	$\mathbf{d}_5, \mathbf{d}_6$	0.0		

Table 4.4-1

Split By Feature	Level	Part.	Instances	Partition Entropy	Rem.	Info. Gain
STREAM	'true'	$\mathcal{D}_1$	$\mathbf{d}_2, \mathbf{d}_3, \mathbf{d}_6, \mathbf{d}_7$	1.5	1.2507	0.3060
	'false'	$\mathcal{D}_2$	$\mathbf{d}_1, \mathbf{d}_4, \mathbf{d}_5$	0.9183		
SLOPE	'flat'	$\mathcal{D}_3$	$\mathbf{d}_5$	0	0.9793	0.5774
	'moderate'	$\mathcal{D}_4$	$\mathbf{d}_2$	0		
	'steep'	$\mathcal{D}_5$	$\mathbf{d}_1, \mathbf{d}_3, \mathbf{d}_4, \mathbf{d}_6, \mathbf{d}_7$	1.3710		

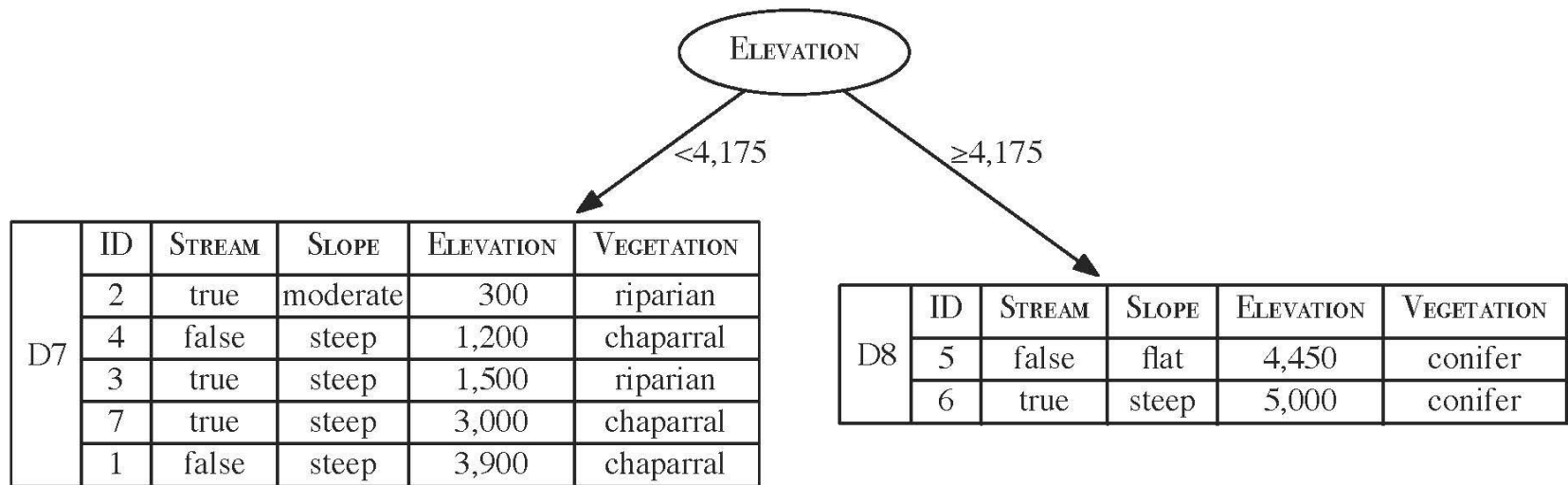


Figure 4.13: The vegetation classification decision tree after the dataset has been split using **ELEVATION** ≥ 4175 .

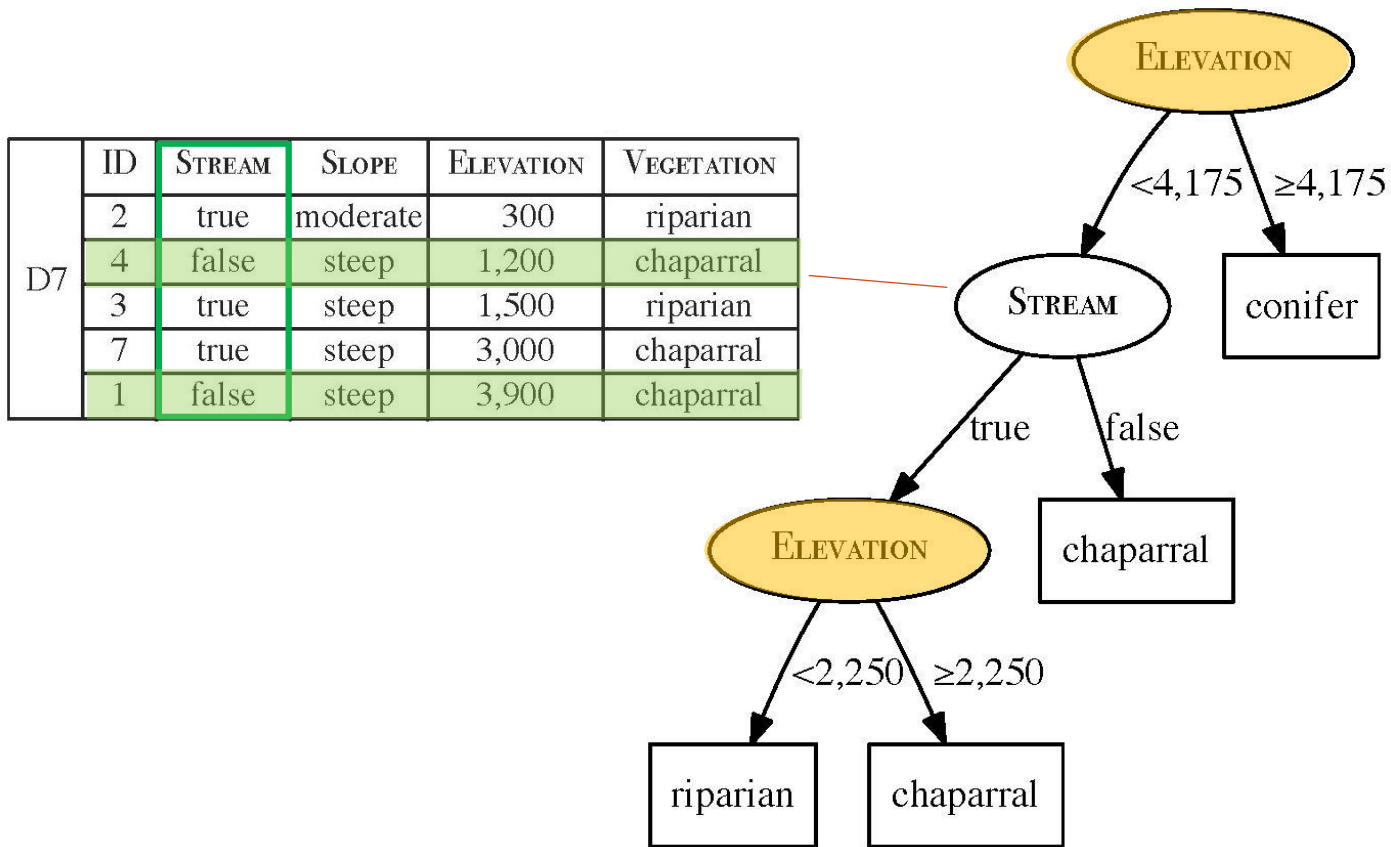


Figure 4.14: The decision tree that would be generated for the vegetation classification dataset listed in Table 4.9 using information gain.

- Unlike categorical features, continuous features can be used at **multiple** points along a path in a decision tree, although the **threshold** applied to the feature at each of these tests will be **different**.
- This is important as it allows **multiple splits** within a range of a continuous feature to be considered on a path.
- Consequently, as we build the rest of the tree, we may reuse the ELEVATION feature.

CONTENTS

- **Alternative Feature Selection Metrics**
- **Handling Continuous Descriptive Features**
- **Predicting Continuous Targets**
- **Noisy Data, Overfitting and Tree Pruning**
- **Model Ensembles**
 - Bagging
 - Boosting
- **Summary**

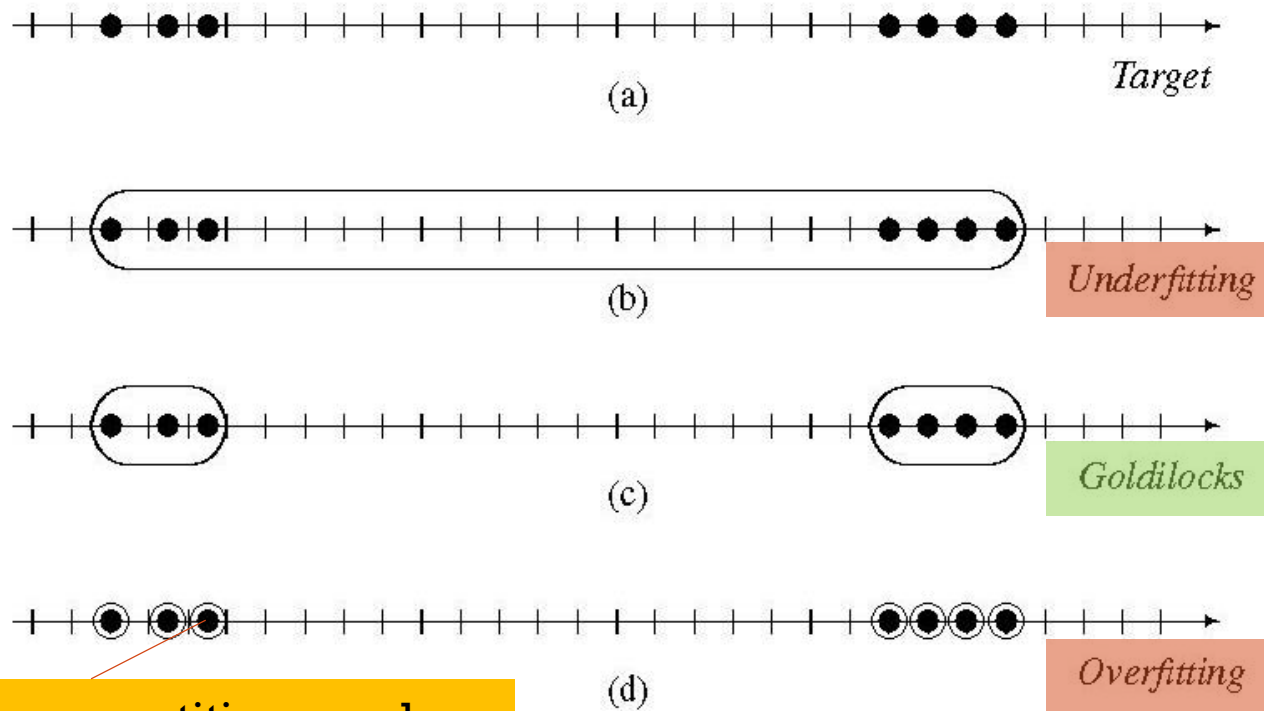
- **Regression trees** are constructed so as to reduce **the variance** in the set of training examples at each of the leaf nodes in the tree
- We can do this by adapting the **ID3 algorithm** to use a measure of **variance** rather than a measure of classification impurity (**entropy**) when selecting the best attribute

- The impurity (variance) at a node can be calculated using the following equation:

$$\text{var}(t, D) = \frac{\sum_{i=1}^n (t_i - \bar{t})^2}{n-1} \quad (4.10)$$

- We select the feature to split on at a node by selecting the feature that **minimizes** the **weighted variance** across the resulting partitions:

$$\mathbf{d}[best] = \arg \min_{d \in \mathbf{d}} \sum_{l \in \text{level}(d)} \frac{|D_{d=l}|}{|D|} \times \text{var}(t, D_{d=l}) \quad (4.11)$$



Although these partitions each have a variance of zero, this is indicative of overfitting the data!

Figure 4.15: (a) A set of instances on a continuous number line; (b), (c), and (d) depict some of the potential groupings that could be applied to these instances.

Algorithm 4.1 Pseudocode description of the ID3 algorithm**Require:** set of descriptive features $\mathbf{d}$ **Require:** set of training instances $\mathcal{D}$

- 1: **if** all the instances in $\mathcal{D}$ have the same target level
- 2: **return** a decision tree consisting of a leaf node
- 3: **else if** $\mathbf{d}$ is empty **then**
- 4: **return** a decision tree consisting of a leaf node with the label of the majority target level in $\mathcal{D}$
- 5: **else if** $\mathcal{D}$ is empty **then**
- 6: **return** a decision tree consisting of a leaf node with the majority target level of the dataset of the immediate parent
- 7: **else**
- 8: $\mathbf{d}[\text{best}] \leftarrow \arg \max_{d \in \mathbf{d}} IG(d, \mathcal{D})$
- 9: make a new node, $Node_{\mathbf{d}[\text{best}]}$, and label it with the majority target level of the dataset of the immediate parent
- 10: partition $\mathcal{D}$ using $\mathbf{d}[\text{best}]$
- 11: remove $\mathbf{d}[\text{best}]$ from $\mathbf{d}$
- 12: **for** each partition $\mathcal{D}_i$ of $\mathcal{D}$ **do**
- 13: grow a branch from $Node_{\mathbf{d}[\text{best}]}$ to the decision tree created by rerunning ID3 with $\mathcal{D} = \mathcal{D}_i$

Early stopping criterion

replaces the base case on Line-1 -stop partitioning the dataset if the number of training instances in the partition is less than some threshold, 5%

In Line-4 and Line-6, the algorithm will create a leaf node that returns the **mean value** of the target feature in a data partition

Replace Line-8 with Equation:

$$\mathbf{d}[\text{best}] = \arg \min_{d \in \mathbf{d}} \sum_{l \in \text{level}(d)} \frac{|D_{d=l}|}{|D|} \times \text{var}(t, D_{d=l})$$

Table 4.11: A dataset listing the number of bike rentals per day.

ID	SEASON	WORK DAY	RENTALS
1	winter	false	800
2	winter	false	826
3	winter	true	900
4	spring	false	2 100
5	spring	true	4 740
6	spring	true	4 900
7	summer	false	3 000
8	summer	true	5 800
9	summer	true	6 200
10	autumn	false	2 910
11	autumn	false	2 880
12	autumn	true	2 820

- **Table 4.12:** The partitioning of the dataset in Table 4.11 based on **SEASON** and **WORK DAY** features and the computation of the weighted variance for each partitioning.

Split by Feature	Level	Part.	Instances	$\frac{ \mathcal{D}_{d=l} }{ \mathcal{D} }$	$var(t, \mathcal{D})$	Weighted Variance
SEASON	'winter'	$\mathcal{D}_1$	$\mathbf{d}_1, \mathbf{d}_2, \mathbf{d}_3$	0.25	2 692	1 379 331
	'spring'	$\mathcal{D}_2$	$\mathbf{d}_4, \mathbf{d}_5, \mathbf{d}_6$	0.25	2 472 533	
	'summer'	$\mathcal{D}_3$	$\mathbf{d}_7, \mathbf{d}_8, \mathbf{d}_9$	0.25	3 040 000	
	'autumn'	$\mathcal{D}_4$	$\mathbf{d}_{10}, \mathbf{d}_{11}, \mathbf{d}_{12}$	0.25	2 100	
WORK DAY	'true'	$\mathcal{D}_5$	$\mathbf{d}_3, \mathbf{d}_5, \mathbf{d}_6, \mathbf{d}_8, \mathbf{d}_9, \mathbf{d}_{12}$	0.50	4 026 346	2 551 813
	'false'	$\mathcal{D}_6$	$\mathbf{d}_1, \mathbf{d}_2, \mathbf{d}_4, \mathbf{d}_7, \mathbf{d}_{10}, \mathbf{d}_{11}$	0.50	1 077 280	

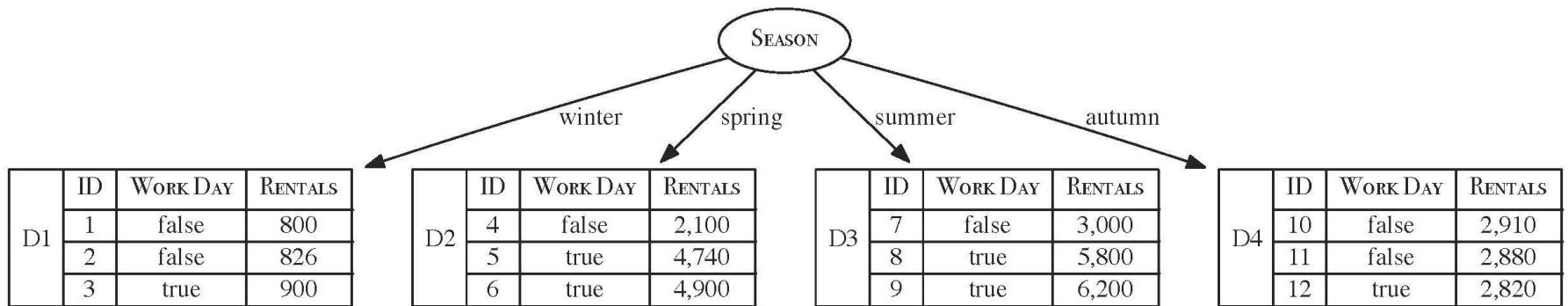


Figure 4.16: The decision tree resulting from splitting the data in Table 4.11 using the feature **SEASON**.

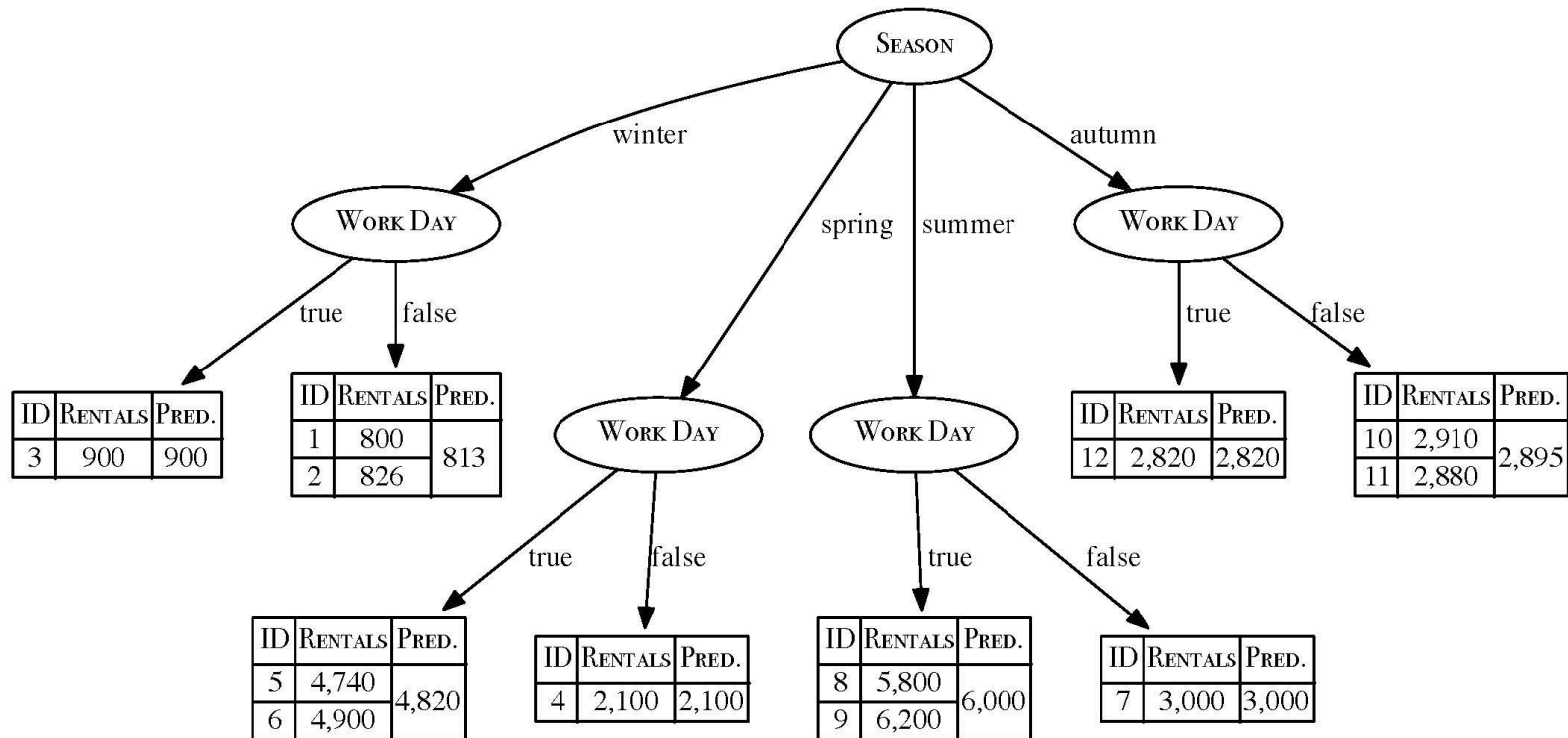


Figure 4.17: The final decision tree induced from the dataset in Table 4.11. To illustrate how the tree generates predictions, this tree lists the instances that ended up at each leaf node and **the prediction (PRED.)** made by each leaf node.

CONTENTS

- **Alternative Feature Selection Metrics**
- **Handling Continuous Descriptive Features**
- **Predicting Continuous Targets**
- **Noisy Data, Overfitting and Tree Pruning**
- **Model Ensembles**
 - Bagging
 - Boosting
- **Summary**

- In the case of a decision tree, **over-fitting** involves splitting the data on an irrelevant feature

*The likelihood of over-fitting occurring increases as a tree gets **deeper** because the resulting classifications are based on **smaller and smaller subsets** as the dataset is partitioned after each feature test in the path.*

Tree pruning identifies and removes subtrees within a decision tree that are likely to be due to **noise** and **sample variance** in the training set used to induce it

- Pruning will result in decision trees being created that are **not consistent** with the training set used to build them
- However, we are more interested in creating prediction models that **generalize well to new data** rather than that are strictly consistent with training data
- It is common to sacrifice consistency for **generalization capacity**.
- Two types: Pre-pruning, Post-pruning

Pre-pruning: stop the recursive partitioning early. Pre-pruning is also known as **forward pruning**.

▪ Common Pre-pruning Approaches

1. early stopping

Strategies:

- the number of instances in a partition falls below a threshold
- the information gain measured at a node is not deemed to be sufficient to make partitioning the data worthwhile
- the depth of the tree goes beyond a predefined limit
-

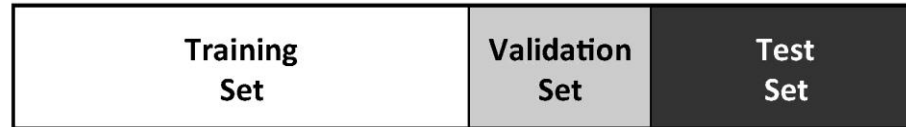
2. χ^2 pruning (*chi-squared*)

- use statistical significance tests to determine the importance of subtrees

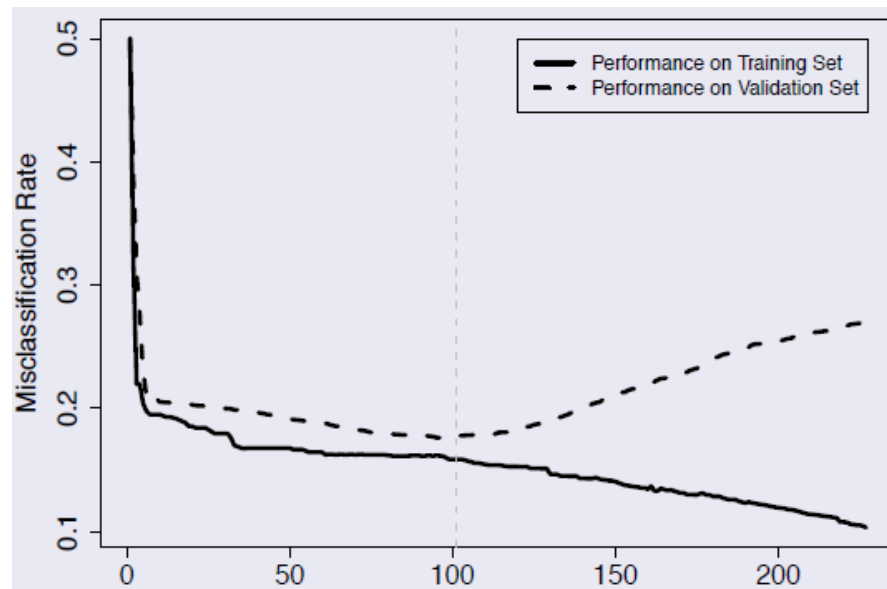
- **Pre-pruning** approaches are **computationally efficient** and can work well for **small datasets**.
- However, induction algorithms that use pre-pruning can **fail to create the most effective trees** because they miss interactions between features that emerge within subtrees that are not obvious when the parent nodes are being considered.

- **Post-pruning**: allow the algorithm to grow the tree as much as it likes and then prune the tree of the branches that cause over-fitting.
- Post-pruning relies on **a criteria** that can distinguish between subtrees that model **relevant aspects** of the data and subtrees that model **irrelevant random patterns** in the data.
- Criteria that compare the **error rate** in the predictions made by a decision tree when a given subtree is included and when it is pruned is recommended.

- Some of the training data is set aside as a **validation dataset**. It is not used during tree induction.



- Using the **validation set** evaluate the prediction accuracy achieved by both the **fully grown** tree and the **pruned copy** of the tree.
- If the pruned copy of the tree performs no worse than the fully grown tree the node is a candidate for pruning.



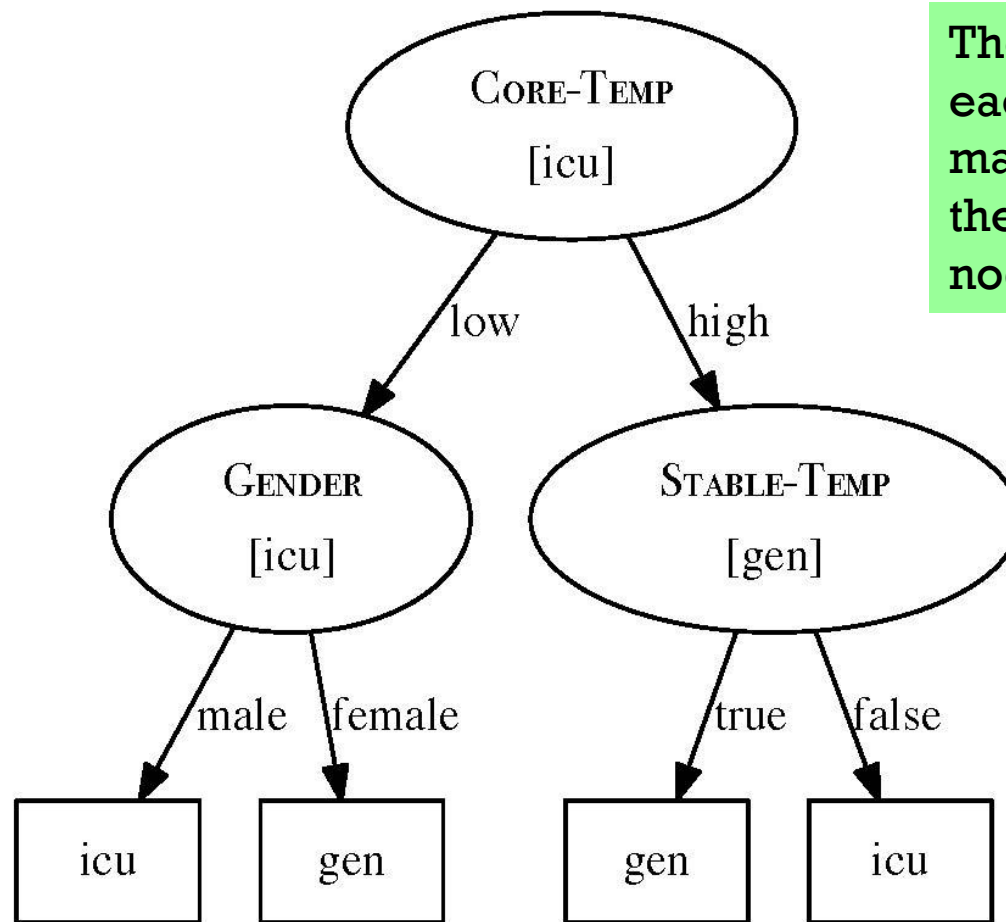
Reduced error pruning (Quinlan, 1987) is a popular version of post-pruning based on error rates.

- A decision tree is built to completion and then the tree is searched in an **iterative, bottom-up, left-to-right** manner for subtrees that can be pruned.
- The error rate resulting from predictions for the instances in the **validation dataset** made **at the root node** of each subtree is compared to the error rate resulting from predictions made **at the leaves** of the subtree.
- If the error rate at the subtree root node is **less than or equal** to the combined error rate at the leaves, the subtree is pruned.

Table 4.13: An example **validation set** for the post-operative patient routing task.

ID	CORE-TEMP	STABLE-TEMP	GENDER	DECISION	
1	high	true	male	gen	general ward
2	low	true	female	icu	
3	high	false	female	icu	intensive care unit
4	high	false	male	icu	
5	low	false	female	icu	
6	low	true	male	icu	

Hypothermia is a major concern for post-operative patients, so many of the descriptive features relevant to this domain relate to a patient's body temperature.



The target level in [] at each interior shows the majority target level for the data partition at that node

Figure 4.18: The decision tree for the post-operative patient routing task.

ID	CORE-TEMP	STABLE-TEMP	GENDER	DECISION
1	high	true	male	gen
2	low	true	female	icu
3	high	false	female	icu
4	high	false	male	icu
5	low	false	female	icu
6	low	true	male	icu

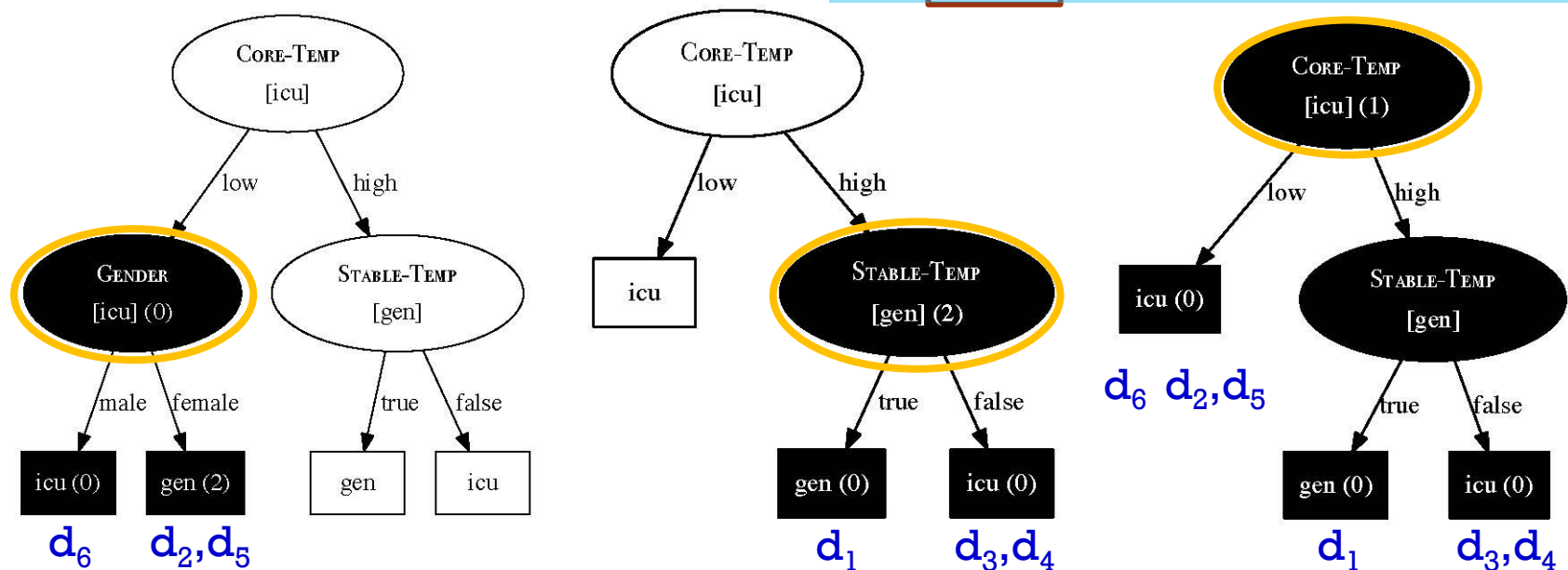


Figure 4.19: The iterations of reduced error pruning for the decision tree in Figure 4.18 using the validation set in Table 4.13.

The subtree that is being considered for pruning in each iteration is highlighted in black. The prediction returned by each non-leaf node is listed in []. The error rate for each leaf node is given in ().

Advantages of pruning:

- It keeps trees smaller, which in turn makes them easier to interpret.
- Increased generalization accuracy when there is noise in the training data (**noise dampening**).

CONTENTS

- **Alternative Feature Selection Metrics**
- **Handling Continuous Descriptive Features**
- **Predicting Continuous Targets**
- **Noisy Data, Overfitting and Tree Pruning**
- **Model Ensembles**
 - Bagging
 - Boosting
- **Summary**

- Rather than creating a **single** model, a **set of** models is generated and then predictions are made by **aggregating** the outputs of these models.
- A prediction model that is composed of a set of models is called a **model ensemble**.
- In order for this approach to work, the models that are in the ensemble must be **different** from each other.

Two **defining characteristics** of ensemble models:

1. Multiple different models are built from **the same dataset** by inducing each model using a **modified version** of the dataset.
2. A prediction is made by aggregating the predictions of the different models in the ensemble.
 - For **categorical** target features, this can be done using different types of **voting mechanisms**
 - For **continuous** target features, this can be done using a measure of the **central tendency** of the different model predictions, such as the **mean** or the **median**.

Exercise 4.7-9: Calculate the probability of a model ensemble that uses simple **majority voting** making an incorrect prediction in the following scenario: The ensemble contains **11 independent models**, all of which have an **error rate of 0.2**.

The ensemble model will make an incorrect prediction for a query when a majority, in this case **6 or more**, of the individual models make an incorrect prediction.

Because each model in the ensemble is **independent** of the others, the predictions returned by the individual models can be viewed as a **sequence of independent binary experiments**.

We can calculate the probability of getting k outcomes, each with a probability of p , in a sequence of n experiments using the binomial distribution as:

$$\binom{n}{k} p^k (1-p)^{n-k}$$

$$\sum_{k=6}^{11} \binom{11}{k} 0.2^k (1-0.2)^{11-k} = 0.017$$

- There are two standard approaches to creating ensembles:
 1. **bagging**
 2. **boosting**

1. BAGGING

- When we use **bagging** (or **bootstrap aggregating**) each model in the ensemble is trained on a random sample of the dataset known as **bootstrap samples**.
- Each random sample is **the same size*** as the dataset and **sampling with replacement** is used.
- Consequently, **each bootstrap sample will be different** and this means that models trained on different bootstrap samples will also be different.

*If we have a very large dataset, we may want to create bootstrap samples that are smaller than the original dataset. In this case, sampling without replacement is preferred. This is known as **subagging**.

1. BAGGING

- **Decision tree** induction algorithms are particularly **well suited** to use with bagging because they are very **sensitive to changes** in the dataset.
- When bagging is used with decision trees, each bootstrap sample only uses a randomly selected **subset of the descriptive features** in the dataset. This is known as **subspace sampling**.
- The combination of **bagging, subspace sampling,** and **decision trees** is known as a **random forest** model.

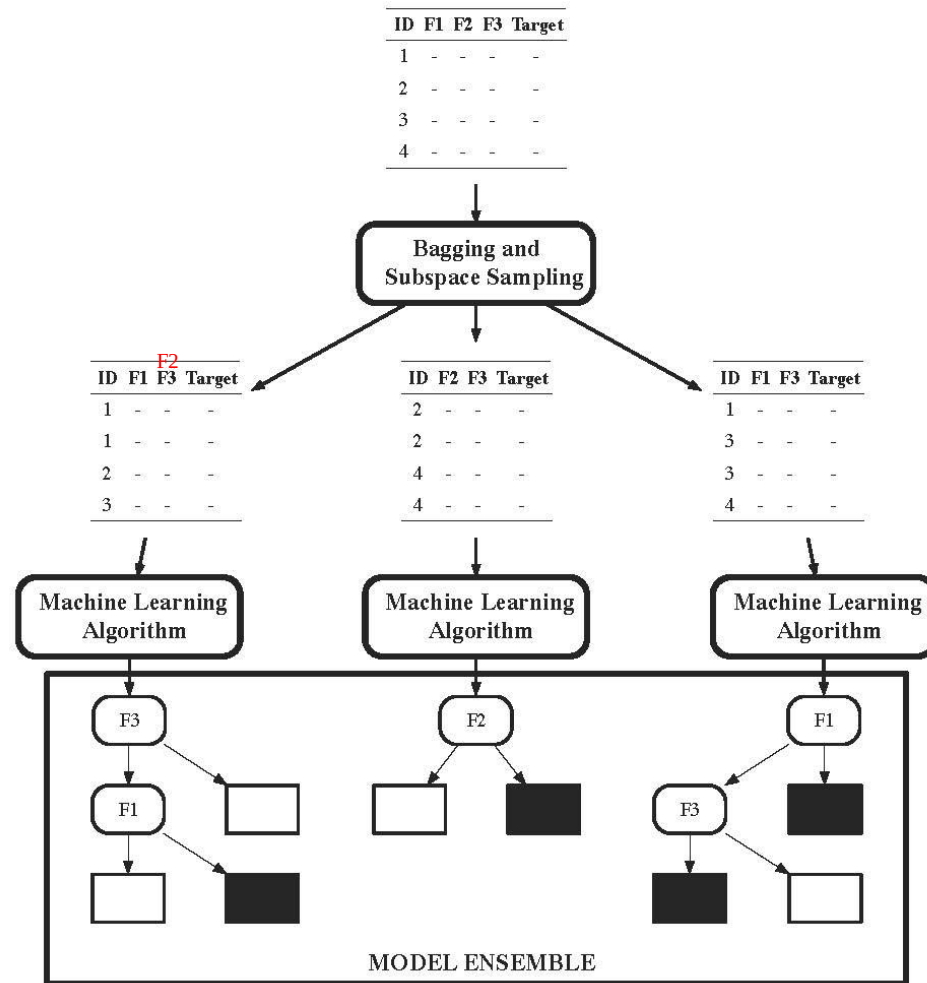


Figure 4.20: The process of creating a model ensemble using bagging and subspace sampling.

- **Exercise 4.7-5:** The following table lists a dataset containing the details of five participants in a heart disease study, and a target feature RISK, which describes their **risk of heart disease**. Each patient is described in terms of four binary descriptive features
 - EXERCISE, how regularly do they exercise
 - SMOKER, do they smoke
 - OBESE, are they overweight
 - FAMILY, did any of their parents or siblings suffer from heart disease

ID	EXERCISE	SMOKER	OBESE	FAMILY	RISK
1	daily	false	false	yes	low
2	weekly	true	false	yes	high
3	daily	false	false	no	low
4	rarely	true	true	yes	high
5	rarely	true	true	no	high

(a) As part of the study, researchers have decided to create a predictive model to screen participants based on their risk of heart disease. You have been asked to implement this screening model using a **random forest**. The three tables below list three **bootstrap samples** that have been generated from the above dataset. Using these bootstrap samples, create the decision trees that will be in the random forest model (use entropy-based information gain as the feature selection criterion).

ID	EXERCISE	FAMILY	RISK
1	daily	yes	low
2	weekly	yes	high
2	weekly	yes	high
5	rarely	no	high
5	rarely	no	high

Bootstrap Sample A

ID	SMOKER	OBESE	RISK
1	false	false	low
2	true	false	high
2	true	false	high
4	true	true	high
5	true	true	high

Bootstrap Sample B

ID	OBESE	FAMILY	RISK
1	false	yes	low
1	false	yes	low
2	false	yes	high
4	true	yes	high
5	true	no	high

Bootstrap Sample C

- The entropy calculation for Bootstrap Sample A is:

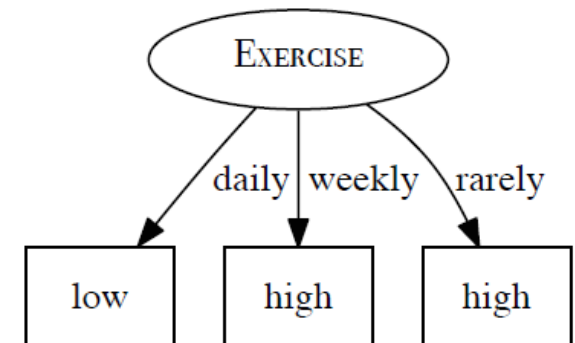
$$\begin{aligned}
 & H(\text{RISK}, \text{BootstrapSampleA}) \\
 &= - \sum_{l \in \{\text{low}, \text{high}\}} P(\text{RISK} = l) \times \log_2(P(\text{RISK} = l)) \\
 &= - \left(\left(\frac{1}{5} \times \log_2 \left(\frac{1}{5} \right) \right) + \left(\frac{4}{5} \times \log_2 \left(\frac{4}{5} \right) \right) \right) \\
 &= 0.7219 \text{ bits}
 \end{aligned}$$

ID	EXERCISE	FAMILY	RISK
1	daily	yes	low
2	weekly	yes	high
2	weekly	yes	high
5	rarely	no	high
5	rarely	no	high

Bootstrap Sample A

- The information gain for each of the features in Bootstrap Sample A is as follows:

Split by Feature	Level	Instances	Partition Entropy	Rem.	Info. Gain
EXERCISE	daily	d ₁	0	0	0.7219
	weekly	d ₂ , d ₂	0		
	rarely	d ₅ , d ₅	0		
FAMILY	yes	d ₁ , d ₂ , d ₂	0.9183	0.5510	0.1709
	no	d ₅ , d ₅	0		



- The entropy calculation for Bootstrap Sample B is:

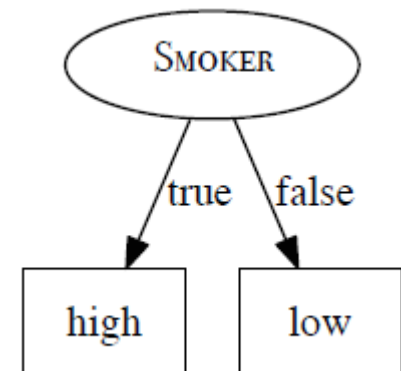
$$\begin{aligned}
 & H(\text{RISK}, \text{BootstrapSampleB}) \\
 &= - \sum_{l \in \{\text{low}, \text{high}\}} P(\text{RISK} = l) \times \log_2 (P(\text{RISK} = l)) \\
 &= - \left(\left(\frac{1}{5} \times \log_2 \left(\frac{1}{5} \right) \right) + \left(\frac{4}{5} \times \log_2 \left(\frac{4}{5} \right) \right) \right) \\
 &= 0.7219 \text{ bits}
 \end{aligned}$$

ID	SMOKER	OBESE	RISK
1	false	false	low
2	true	false	high
2	true	false	high
4	true	true	high
5	true	true	high

Bootstrap Sample B

- The information gain for each of the features in Bootstrap Sample B is as follows:

Split by Feature	Level	Instances	Partition Entropy	Rem.	Info. Gain
SMOKER	true	d ₂ , d ₂ , d ₄ , d ₅	0	0	0.7219
	false	d ₁	0		
OBESE	true	d ₄ , d ₅	0	0.5510	0.1709
	false	d ₁ , d ₂ , d ₂	0.9183		



- The entropy calculation for Bootstrap Sample C is:

$$\begin{aligned}
 &H(\text{RISK}, \text{BootstrapSampleC}) \\
 &= - \sum_{l \in \{\text{low}, \text{high}\}} P(\text{RISK} = l) \times \log_2(P(\text{RISK} = l)) \\
 &= - \left(\left(\frac{2}{5} \times \log_2 \left(\frac{2}{5} \right) \right) + \left(\frac{3}{5} \times \log_2 \left(\frac{3}{5} \right) \right) \right) \\
 &= 0.9710 \text{ bits}
 \end{aligned}$$

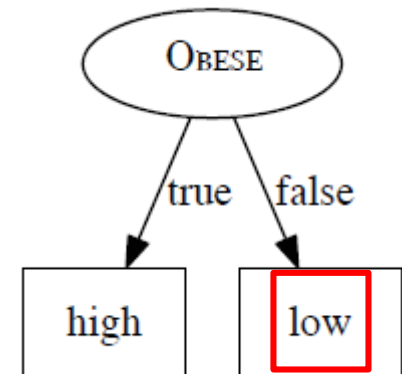
majority

ID	OBESE	FAMILY	RISK
1	false	yes	low
1	false	yes	low
2	false	yes	high
4	true	yes	high
5	true	no	high

Bootstrap Sample C

- The information gain for each of the features in Bootstrap Sample C is as follows:

Split by Feature	Level	Instances	Partition Entropy	Rem.	Info. Gain
OBESE	true	d ₄ , d ₅	0	0.5510	0.4200
	false	d ₁ , d ₁ , d ₂	0.9183		
FAMILY	yes	d ₁ , d ₁ , d ₂ , d ₄	1.0	0.8	0.1709
	no	d ₅	0		



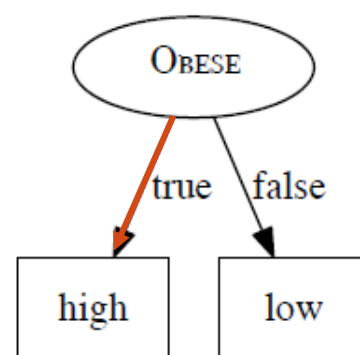
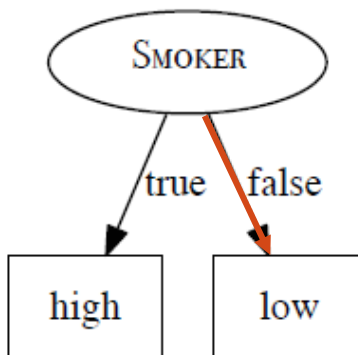
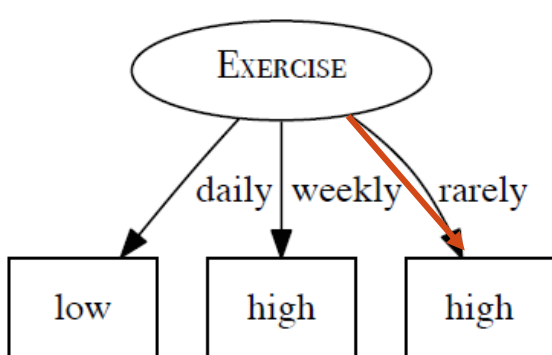
(b) Assuming the random forest model you have created uses majority voting, what prediction will it return for the following query:

EXERCISE=*rarely*, SMOKER=*false*, OBESE=*true*, FAMILY=*yes*

Each of the trees in the ensemble will vote as follows:

- Tree 1: EXERCISE=*rarely* → RISK=*high*
- Tree 2: SMOKER=*false* → RISK=*low*
- Tree 3: OBESE=*true* → RISK=*high*

So, the **majority vote** is for RISK=*high*, and this is the prediction the model will return for this query.



2. BOOSTING

- Boosting works by **iteratively** creating models and adding them to the ensemble.
- The iteration stops when a predefined number of models have been added.
- When we use **boosting**, each new model added to the ensemble is **biased to pay more attention** to instances that previous models **miss-classified**.
- This is done by **incrementally adapting** the dataset used to train the models. To do this we use a **weighted dataset**

2. BOOSTING

Weighted Dataset

- Each instance has an associated weight $w_i \geq 0$,
- Initially set to $1/n$ where n is the number of instances in the dataset.
- After each model is added to the ensemble, it is tested on the training data and the weights of the instances the model gets **correct** are **decreased** and the weights of the instances the model gets **incorrect** are **increased**.
- These **weights are used as a distribution** over which the dataset is sampled to create a **replicated training set**, where the replication of an instance is proportional to its weight.

2. BOOSTING

During each training iteration the algorithm:

1. Induces a model and calculates the total error, ε , by **summing the weights** of the training instances for which the predictions made by the model are **incorrect**.
2. Increases the weights for the instances misclassified using:

$$w[i] \leftarrow w[i] \times \left(\frac{1}{2 \times \varepsilon} \right) \quad (4.12)$$

3. Decreases the weights for the instances correctly classified:

$$w[i] \leftarrow w[i] \times \left(\frac{1}{2 \times (1 - \varepsilon)} \right) \quad (4.13)$$

4. Calculate a **confidence factor**, α , for the model such that α increases as ε decreases:

$$\alpha = \frac{1}{2} \times \log_e \left(\frac{1 - \varepsilon}{\varepsilon} \right) \quad (4.14)$$

2. BOOSTING

- Once the set of models have been created, the ensemble makes predictions using a **weighted aggregate** of the predictions made by the individual models.
- The weights used in this aggregation are simply the **confidence factors** associated with each model
 - For categorical target features, the ensemble returns the majority target level using a **weighted vote**
 - For continuous target features, the ensemble returns the **weighted mean**

- **Table 4.14:** A simple bicycle demand predictions dataset and the workings of the first three iterations of training an ensemble model using boosting to predict RENTALS given TEMP.

ID	TEMP	RENTALS	Iteration 0			Iteration 1			Iteration 2		
			Dist.	Freq.	$M_0(d)$	Dist.	Freq.	$M_1(d)$	Dist.	Freq.	$M_2(d)$
1	4	Low	0.100	2	Low	0.062	0	High	0.167	2	Low
2	5	Low	0.100	1	Low	0.062	1	High	0.167	1	Low
3	7	Low	0.100	0	Low	0.062	1	High	0.167	3	Low
4	12	High	0.100	1	High	0.062	2	High	0.038	0	Low
5	18	High	0.100	1	High	0.062	0	High	0.038	0	Low
6	23	High	0.100	1	High	0.062	0	High	0.038	0	Low
7	27	High	0.100	1	High	0.062	1	High	0.038	0	Low
8	28	High	0.100	1	High	0.062	1	High	0.038	1	Low
9	32	Low	0.100	2	High	0.250	3	Low	0.154	1	Low
10	35	Low	0.100	0	High	0.250	1	Low	0.154	2	Low

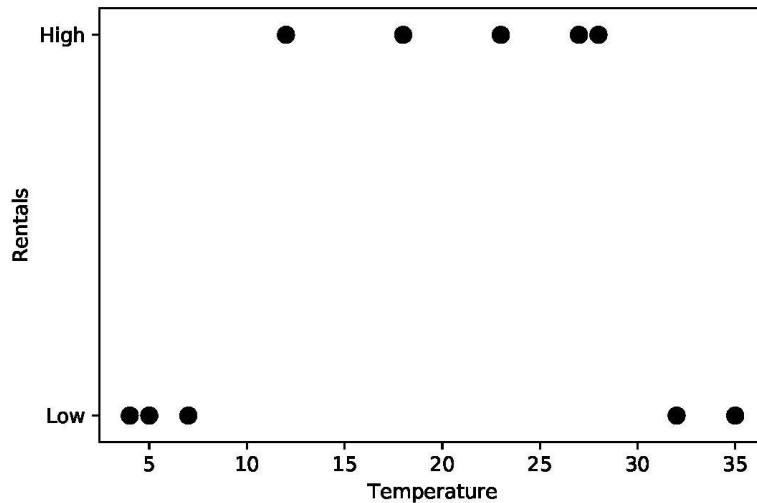
$$w_1[1] \leftarrow 0.100 \times \left(\frac{1}{2 \times (1 - 0.200)} \right) = 0.0625$$

$$w_1[9] \leftarrow 0.100 \times \left(\frac{1}{2 \times 0.200} \right) = 0.250$$

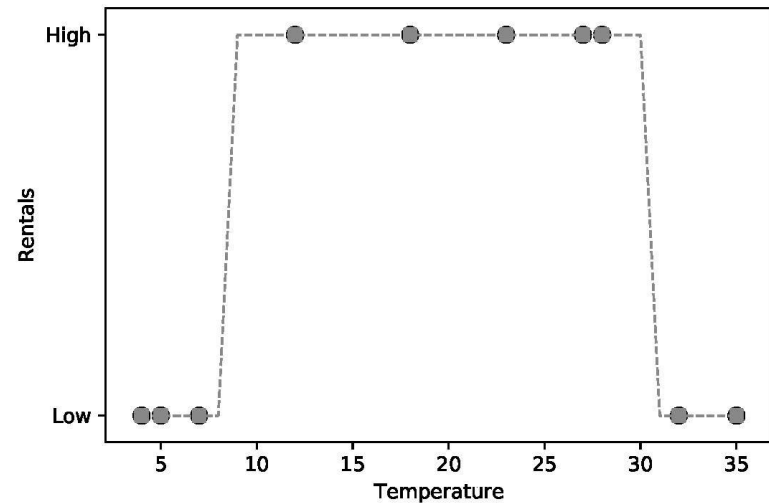
$$w_2[1] \leftarrow 0.062 \times \left(\frac{1}{2 \times 0.062 \times 3} \right) = 0.167$$

$$w_2[4] \leftarrow 0.062 \times \left(\frac{1}{2 \times (1 - 0.062 \times 3)} \right) = 0.038$$

$$w_2[9] \leftarrow 0.250 \times \left(\frac{1}{2 \times (1 - 0.062 \times 3)} \right) = 0.154$$

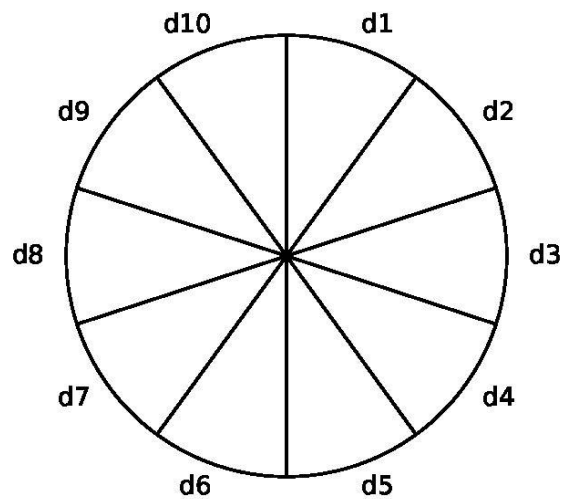


(a) Training data

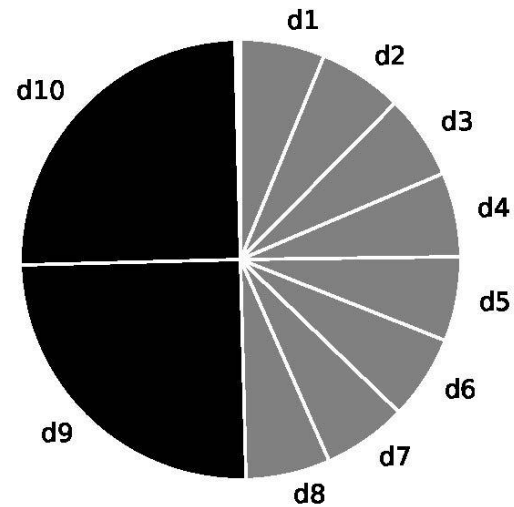


(b) The final ensemble model, M

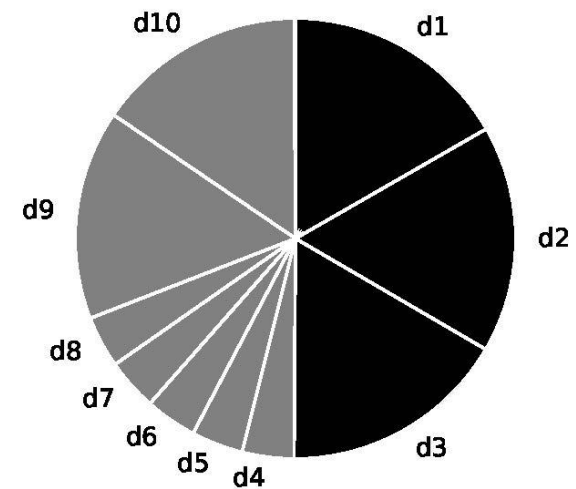
Figure 4.21: (a) A plot of the bike rental dataset from Table 4.14. (b) An illustration of the final ensemble model trained using 5 iterations of the boosting algorithm.



(c) Distribution 0



(d) Distribution 1



(e) Distribution 2

Figure 4.21: (c)–(e) A representation of the changing weights used to generate sample datasets for the first iterations of the boosting process.

Exercise 4.7-11: The following table shows a set of predictions made by six models in an ensemble and the ground truth of the target feature in a small **test dataset**, PROGNOSIS.

ID	PROGNOSIS	M ₀	M ₁	M ₂	M ₃	M ₄	M ₅
1	<i>Bad</i>	<i>Bad</i>	<i>Bad</i>	<i>Good</i>	<i>Bad</i>	<i>Bad</i>	<i>Good</i>
2	<i>Good</i>	<i>Good</i>	<i>Good</i>	<i>Good</i>	<i>Bad</i>	<i>Good</i>	<i>Bad</i>
3	<i>Good</i>	<i>Bad</i>	<i>Good</i>	<i>Bad</i>	<i>Good</i>	<i>Good</i>	<i>Good</i>
4	<i>Bad</i>	<i>Bad</i>	<i>Bad</i>	<i>Bad</i>	<i>Bad</i>	<i>Bad</i>	<i>Good</i>
5	<i>Bad</i>	<i>Good</i>	<i>Bad</i>	<i>Good</i>	<i>Bad</i>	<i>Good</i>	<i>Good</i>

(a) Assuming that these models are part of an ensemble training using **bagging**, calculate the overall output of the ensemble for each instance in the test dataset.

ID	PROGNOSIS	<i>Bad</i> Votes	<i>Good</i> Votes	M
1	<i>Bad</i>	4	2	<i>Bad</i>
2	<i>Good</i>	2	4	<i>Good</i>
3	<i>Good</i>	2	4	<i>Good</i>
4	<i>Bad</i>	5	1	<i>Bad</i>
5	<i>Bad</i>	2	4	<i>Good</i>

(b) Measure the performance of this bagged ensemble using **misclassification rate**.

The ensemble model only made an **incorrect** prediction for **1** out of 5 instances in the test dataset .

The misclassification rate is $1/5=20\%$.

(c) Assuming that these models are part of an ensemble trained using **boosting** and that the **confidence factors** α , for the models are as follows, calculate the overall output of the ensemble for each instance in the **test dataset**.

		M_0	M_1	M_2	M_3	M_4	M_5
		0.114	0.982	0.653	0.912	0.883	0.233
ID	PROGNOSIS	M_0	M_1	M_2	M_3	M_4	M_5
1	<i>Bad</i>	<i>Bad</i>	<i>Bad</i>	<i>Good</i>	<i>Bad</i>	<i>Bad</i>	<i>Good</i>
2	<i>Good</i>	<i>Good</i>	<i>Good</i>	<i>Good</i>	<i>Bad</i>	<i>Good</i>	<i>Bad</i>
3	<i>Good</i>	<i>Bad</i>	<i>Good</i>	<i>Bad</i>	<i>Good</i>	<i>Good</i>	<i>Good</i>
4	<i>Bad</i>	<i>Bad</i>	<i>Bad</i>	<i>Bad</i>	<i>Bad</i>	<i>Bad</i>	<i>Good</i>
5	<i>Bad</i>	<i>Good</i>	<i>Bad</i>	<i>Good</i>	<i>Bad</i>	<i>Good</i>	<i>Good</i>

In the boosting case, the votes are **weighted** by the confidence factor associated with each model.

For d_1 , the weighted vote for **Bad** is $0.114 + 0.982 + 0.912 + 0.883 = 2.891$, the weighted vote for **Good** is $0.653 + 0.233 = 0.886$.

The table below shows these weighted votes.

ID	PROGNOSIS	<i>Bad</i>	<i>Good</i>	M
		Votes	Votes	
1	<i>Bad</i>	2.891	0.886	<i>Bad</i>
2	<i>Good</i>	1.145	2.632	<i>Good</i>
3	<i>Good</i>	0.767	3.01	<i>Good</i>
4	<i>Bad</i>	3.544	0.233	<i>Bad</i>
5	<i>Bad</i>	1.894	1.883	<i>Bad</i>

The last instance, $\mathbf{d}_5$ is particularly interesting in this case.

Although 4 of the 6 models have made a prediction of **Good**, the 2 models with the highest confidence factors have made a prediction of **Bad** and the strength of their votes outweighs the others.

In the case the ensemble model made the correct prediction for all instances in the test dataset and so the misclassification rate is 0%.

2.1 GRADIENT BOOSTING

- Gradient boosting is more recently developed, and very effective.
- It also **iteratively** trains prediction models, trying to make later models specialize in areas that earlier models **struggled with** in a **more aggressive way**.
- Later models are trained to **directly correct errors** made by earlier models, rather than the more **subtle** approach of simply changing weights in a sampling distribution to **encourage** this.

2.1 GRADIENT BOOSTING

- Gradient boosting algorithm begins by training a **very simple base model**, M_0 .
- For problems with a continuous target, this model typically simply predicts the overall average target value from the training dataset

$$\mathbf{M}_0(\mathbf{d}) = \frac{1}{n} \sum_{i=1}^n t_i \quad (4.15)$$

2.1 GRADIENT BOOSTING

- Gradient boosting then **iteratively** adds new models to the ensemble.
- The newly trained models are directly trying to **correct**, and therefore **improve**, the predictions made by the previous models

$$\mathbf{M}_1(\mathbf{d}) = \mathbf{M}_0(\mathbf{d}) + \mathbf{M}_{\Delta 1}(\mathbf{d}) \quad (4.16)$$

2.1 GRADIENT BOOSTING

- The gradient boosting algorithm proceeds by adding more and more models where each model is trained to improve the prediction of the previous ones.

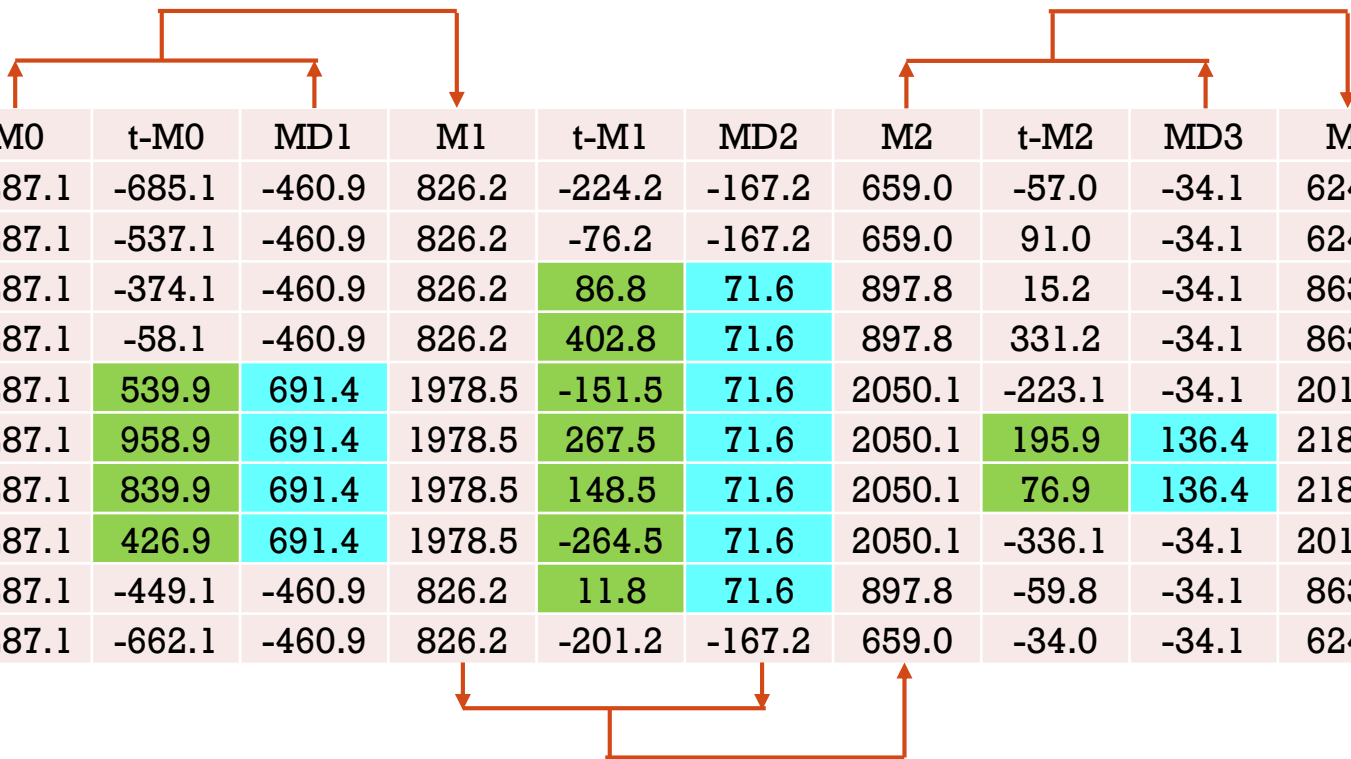
$$\mathbf{M}_i(\mathbf{d}) = \mathbf{M}_{i-1}(\mathbf{d}) + \mathbf{M}_{\Delta i}(\mathbf{d}) \quad (4.17)$$

2.1 GRADIENT BOOSTING

- The final model is then a model that makes a basic prediction and adds a number of improvements to this prediction.
- Consider the model trained after **four iterations**

$$\begin{aligned}\mathbf{M}_4(\mathbf{d}) &= \mathbf{M}_3(\mathbf{d}) + \mathbf{M}_{\Delta 4}(\mathbf{d}) & (4.18) \\ &= (\mathbf{M}_2(\mathbf{d}) + \mathbf{M}_{\Delta 3}(\mathbf{d})) + \mathbf{M}_{\Delta 4}(\mathbf{d}) \\ &= ((\mathbf{M}_1(\mathbf{d}) + \mathbf{M}_{\Delta 2}(\mathbf{d})) + \mathbf{M}_{\Delta 3}(\mathbf{d})) + \mathbf{M}_{\Delta 4}(\mathbf{d}) \\ &= (((\mathbf{M}_0(\mathbf{d}) + \mathbf{M}_{\Delta 1}(\mathbf{d})) + \mathbf{M}_{\Delta 2}(\mathbf{d})) + \mathbf{M}_{\Delta 3}(\mathbf{d})) + \mathbf{M}_{\Delta 4}(\mathbf{d}) \\ &= \mathbf{M}_0(\mathbf{d}) + \mathbf{M}_{\Delta 1}(\mathbf{d}) + \mathbf{M}_{\Delta 2}(\mathbf{d}) + \mathbf{M}_{\Delta 3}(\mathbf{d}) + \mathbf{M}_{\Delta 4}(\mathbf{d})\end{aligned}$$

TABLE 4.15: A SIMPLE BICYCLE DEMAND PREDICTIONS DATASET AND THE WORKINGS OF THE FIRST ITERATIONS OF TRAINING A GRADIENT BOOSTING MODEL.



ID	TEMP	TENT	M0	t-M0	MD1	M1	t-M1	MD2	M2	t-M2	MD3	M3
1	4	602	1287.1	-685.1	-460.9	826.2	-224.2	-167.2	659.0	-57.0	-34.1	624.9
2	5	750	1287.1	-537.1	-460.9	826.2	-76.2	-167.2	659.0	91.0	-34.1	624.9
3	7	913	1287.1	-374.1	-460.9	826.2	86.8	71.6	897.8	15.2	-34.1	863.7
4	12	1229	1287.1	-58.1	-460.9	826.2	402.8	71.6	897.8	331.2	-34.1	863.7
5	18	1827	1287.1	539.9	691.4	1978.5	-151.5	71.6	2050.1	-223.1	-34.1	2016.0
6	23	2246	1287.1	958.9	691.4	1978.5	267.5	71.6	2050.1	195.9	136.4	2186.5
7	27	2127	1287.1	839.9	691.4	1978.5	148.5	71.6	2050.1	76.9	136.4	2186.5
8	28	1714	1287.1	426.9	691.4	1978.5	-264.5	71.6	2050.1	-336.1	-34.1	2016.0
9	32	838	1287.1	-449.1	-460.9	826.2	11.8	71.6	897.8	-59.8	-34.1	863.7
10	35	625	1287.1	-662.1	-460.9	826.2	-201.2	-167.2	659.0	-34.0	-34.1	624.9

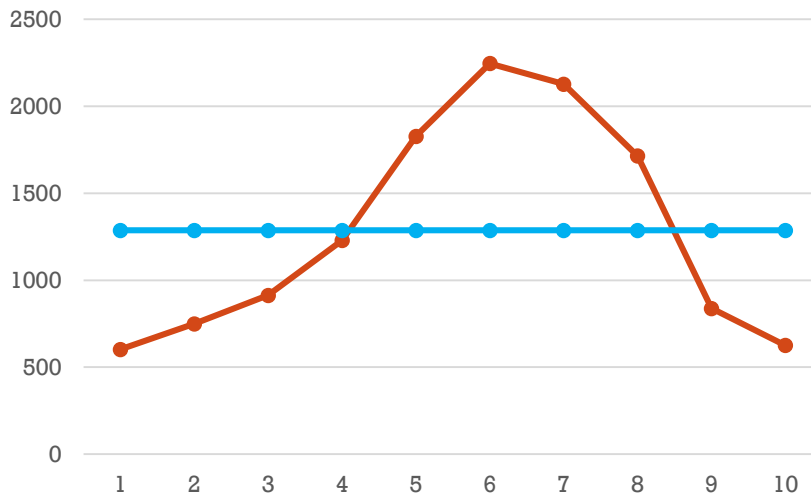
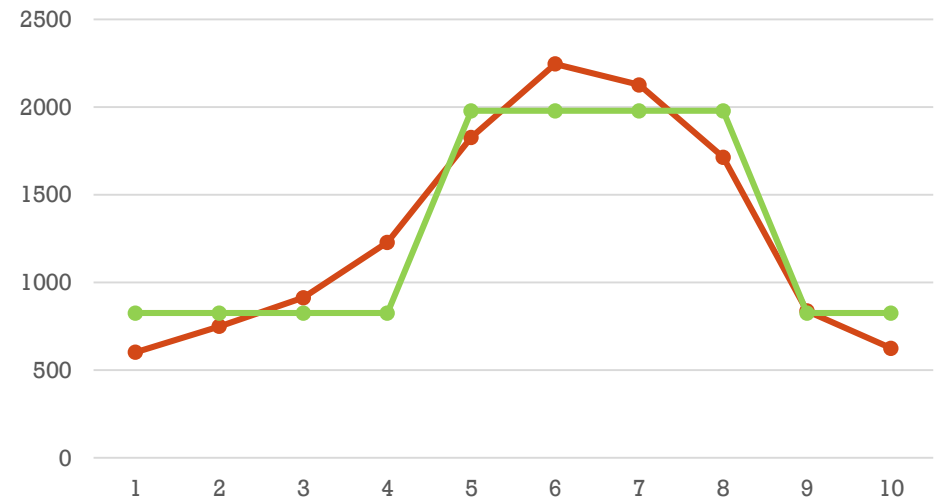
(a) M_0 (b) M_1

Figure 4.22: A plot of the bike rental dataset from Table 4.15, and visualizations of the prediction models trained in the early iterations of the gradient boosting process.

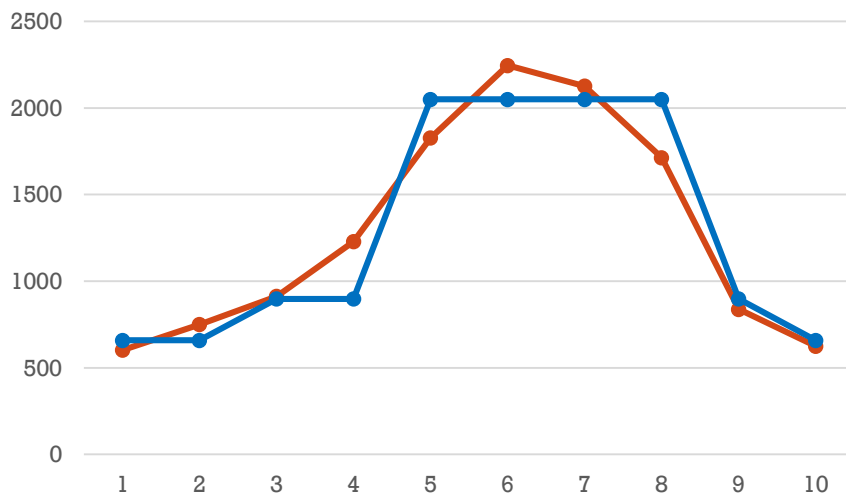
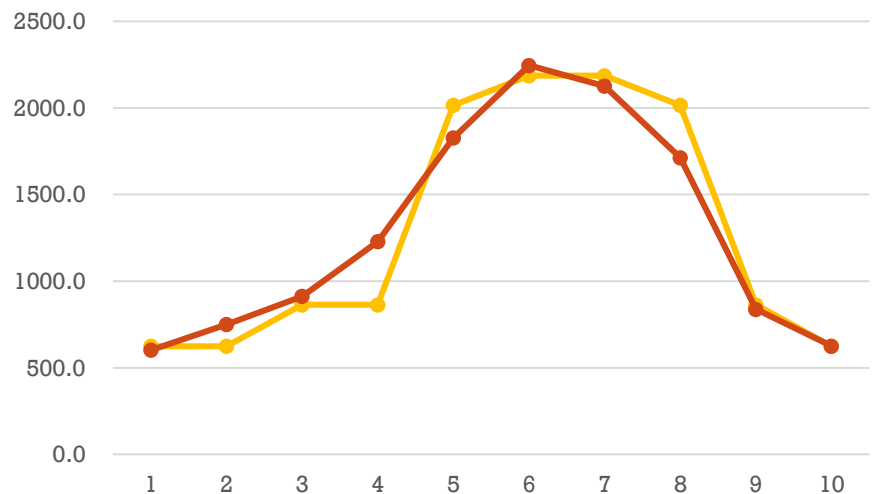
(c) M_2 (d) M_3

Figure 4.22: A plot of the bike rental dataset from Table 4.15, and visualizations of the prediction models trained in the early iterations of the gradient boosting process.

2.1 GRADIENT BOOSTING

- The gradient boosting algorithm described here does not have an **explicit** aggregation step
- Instead, the final model trained provides the overall output of the ensemble as it **implicitly** combines the outputs of all models
- The simple version can be extended in many ways. For example, a **learning rate** α can be added, which makes models more robust to outliers

$$\mathbf{M}_i(\mathbf{d}) = \mathbf{M}_{i-1}(\mathbf{d}) + \alpha \times \mathbf{M}_{\Delta i}(\mathbf{d}) \quad (4.19)$$

2.1 GRADIENT BOOSTING

- The gradient boosting approach can also be easily adapted to work with **categorical targets**.
- In this case, the initial model predicts a **likelihood** that an instance belongs to each of the possible target levels, and subsequent models predict corrections to these likelihoods.

2.1 GRADIENT BOOSTING

- The version of the gradient boosting algorithm described here searches for an ensemble model that **minimizes the mean squared error** between the target feature values from the training dataset and the model predictions.
- It can be extended to learn to **minimize other loss functions**, which makes it an extremely effective and flexible approach to predictive modeling.

3. BOOSTING OR BAGGING

- Which approach should we use?
- Bagging is **simpler** to implement and **parallelize** than boosting and, so, may be better with respect to ease of use and training time.
- Empirical results indicate:
 - **boosted** decision tree ensembles were the best performing model of those tested for datasets containing **up to 4,000** descriptive features.
 - **random forest** ensembles (based on bagging) performed better for datasets containing **more than 4,000** features.
 - boosted ensembles are **prone to overfitting**, and in domains with large numbers of features, overfitting becomes a serious problem.

CONTENTS

- **Alternative Feature Selection Metrics**
- **Handling Continuous Descriptive Features**
- **Predicting Continuous Targets**
- **Noisy Data, Overfitting and Tree Pruning**
- **Model Ensembles**
 - Boosting
 - Bagging
- **Summary**

- The **decision tree** model makes predictions based on sequences of tests on the descriptive feature values of a query
- The **ID3** algorithm as a standard algorithm for inducing decision trees from a dataset.

Decision Trees: **Advantages**

- interpretable
- handle both categorical and continuous descriptive features
- has the ability to model the interactions between descriptive features (diminished if **pre-pruning** is employed)
- relatively, robust to the **curse of dimensionality**
- relatively, robust to noise in the dataset if **pruning** is used.

Decision Trees: **Potential Disadvantages**

- trees become **large** when dealing with continuous features.
- decision trees are very expressive and sensitive to the dataset, as a result they can **overfit** the data if there are a lot of features (curse of dimensionality)
- eager learner
 - not suitable for modeling concepts that change over time (concept drift), because they will need to be retrained

HOMEWORK

- Reading: Section 4.4~4.6

Appendix B

- Exercises: 4.7-3, 7, 8

HOMEWORK

- 10月21日之前通过教学网提交第1~4章习题作业

第1章 1.9-4,9

第2章 2.7-3

第3章 3.9-1,2,5,6

第4章 4.7-3,7,8